

In []:

In []:

```
In [2]: import numpy as np
import cv2
import os
import matplotlib.pyplot as plt
from skimage.metrics import structural_similarity as ssim
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Henon Map for scrambling (permutation)
def henon_map_sequence(size, x0=0.1, y0=0.3, a=1.4, b=0.3):
    x, y = x0, y0
    seq = np.zeros(size, dtype=np.float64)
    for i in range(size):
        x_new = 1 - a * x ** 2 + y
        y_new = b * x
        x, y = x_new, y_new
        seq[i] = abs(x) % 1 # Normalize between 0 and 1
    return seq

# Logistic Map for pixel modification (diffusion)
def logistic_map_sequence(size, r=3.99, x0=0.5):
    x = x0
    seq = np.zeros(size, dtype=np.float64)
    for i in range(size):
        x = r * x * (1 - x)
        seq[i] = abs(x) % 1 # Normalize between 0 and 1
    return seq

# Encrypt image
def encrypt_image(image):
    h, w = image.shape
    total_pixels = h * w

    # Generate Henon and Logistic chaotic sequences
    henon_seq = henon_map_sequence(total_pixels)
    logistic_seq = logistic_map_sequence(total_pixels)

    # Permutation: Sort pixel positions using Henon sequence
    perm_indices = np.argsort(henon_seq)
    flattened_image = image.flatten()
    permuted_image = flattened_image[perm_indices]

    # Diffusion: Modify pixel values using Logistic sequence
    encrypted_image = (permuted_image + (logistic_seq * 255)) % 256

    return encrypted_image.reshape(h, w).astype(np.uint8), perm_indices, logis

# Decrypt image
def decrypt_image(encrypted_image, perm_indices, logistic_seq):
    h, w = encrypted_image.shape
    total_pixels = h * w

    # Reverse diffusion
    encrypted_flat = encrypted_image.flatten()
```

```
diffused_image = (encrypted_flat - (logistic_seq * 255)) % 256

# Reverse permutation
original_image = np.zeros(total_pixels, dtype=np.uint8)
original_image[perm_indices] = diffused_image

return original_image.reshape(h, w)

# Compute SSIM
def compute_ssim(original, decrypted):
    return ssim(original, decrypted, data_range=255)

# Compute NPCR & UACI
def compute_npcr_uaci(original, encrypted):
    diff = original.astype(np.int16) - encrypted.astype(np.int16)
    npcr = np.sum(original != encrypted) / original.size * 100
    uaci = np.sum(np.abs(diff)) / (original.size * 255) * 100
    return npcr, uaci

# Directories
data_dir="C:/Users/hitha sunil/Desktop/DOTA/preprocessed/dota_resized" # Chang
output_encrypted_dir = "C:/Users/hitha sunil/Desktop/DOTA/Chaos/encrypted_data"
output_decrypted_dir = "C:/Users/hitha sunil/Desktop/DOTA/Chaos/decrypted_data"

os.makedirs(output_encrypted_dir, exist_ok=True)
os.makedirs(output_decrypted_dir, exist_ok=True)

image_files = [f for f in os.listdir(data_dir) if f.endswith(('.png', '.jpg',
ssim_scores = []
npcr_values = []
uaci_values = []

features_original = []
features_decrypted = []
labels = []

for img_file in image_files:
    img_path = os.path.join(data_dir, img_file)
    image = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    image = cv2.resize(image, (128, 128)) # Resize for consistency

    encrypted_img, perm_indices, logistic_seq = encrypt_image(image)
    decrypted_img = decrypt_image(encrypted_img, perm_indices, logistic_seq)

    cv2.imwrite(os.path.join(output_encrypted_dir, img_file), encrypted_img)
    cv2.imwrite(os.path.join(output_decrypted_dir, img_file), decrypted_img)

    # Compute SSIM
    score = compute_ssim(image, decrypted_img)
    ssim_scores.append(score)

    # Compute NPCR & UACI
    npcr, uaci = compute_npcr_uaci(image, encrypted_img)
    npcr_values.append(npcr)
    uaci_values.append(uaci)
```

```
# Feature Extraction
features_original.append(image.flatten())
features_decrypted.append(decrypted_img.flatten())
labels.append(1) # Original image label
labels.append(0) # Decrypted image label

print(f"Processed {len(image_files)} images successfully.")
print(f"Average SSIM: {np.mean(ssim_scores):.4f}")
print(f"Average NPCR: {np.mean(npcr_values):.2f}%")
print(f"Average UACI: {np.mean(uaci_values):.2f}%")

# Classification Accuracy
X = np.array(features_original + features_decrypted)
y = np.array(labels)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

classifier = SVC(kernel='linear')
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Classification Accuracy: {accuracy:.4f}")
```

```
Processed 2806 images successfully.
Average SSIM: 0.9878
Average NPCR: 99.41%
Average UACI: 26.27%
Classification Accuracy: 0.9118
```

In []:

In []:

In []:

In []:

```
In [2]: import numpy as np
import cv2
import os
import matplotlib.pyplot as plt
import seaborn as sns
import time
from skimage.metrics import structural_similarity as ssim
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from scipy.spatial.distance import euclidean

# =====
# 1. CHAOTIC MAP DEFINITIONS
# =====

def henon_map_sequence(size, x0=0.1, y0=0.3, a=1.4, b=0.3):
    x, y = x0, y0
    seq = np.zeros(size, dtype=np.float64)
    for i in range(size):
        x_new = 1 - a * x**2 + y
        y_new = b * x
        x, y = x_new, y_new
        seq[i] = abs(x) % 1
    return seq

def logistic_map_sequence(size, r=3.99, x0=0.5):
    x = x0
    seq = np.zeros(size, dtype=np.float64)
    for i in range(size):
        x = r * x * (1 - x)
        seq[i] = abs(x) % 1
    return seq

# =====
# 2. ENCRYPTION & DECRYPTION (REVERSIBLE)
# =====

def encrypt_image(image):
    h, w = image.shape
    total_pixels = h * w

    henon_seq = henon_map_sequence(total_pixels)
    logistic_seq = logistic_map_sequence(total_pixels)

    perm_indices = np.argsort(henon_seq)
    flattened = image.flatten()
    permuted = flattened[perm_indices]

    encrypted = (permuted + (logistic_seq * 255)) % 256

    return encrypted.reshape(h, w).astype(np.uint8), perm_indices, logistic_se
```

```
def decrypt_image(encrypted_image, perm_indices, logistic_seq):
    h, w = encrypted_image.shape
    encrypted_flat = encrypted_image.flatten()

    diffused = (encrypted_flat - (logistic_seq * 255)) % 256

    original = np.zeros_like(diffused)
    original[perm_indices] = diffused

    return original.reshape(h, w).astype(np.uint8)

# =====
# 3. METRICS
# =====

def compute_ssim(original, decrypted):
    return ssim(original, decrypted, data_range=255)

def compute_npcr_uaci(original, encrypted):
    diff = original.astype(np.int16) - encrypted.astype(np.int16)
    npcr = np.sum(original != encrypted) / original.size * 100
    uaci = np.sum(np.abs(diff)) / (original.size * 255) * 100
    return npcr, uaci

def image_entropy(img):
    hist = np.histogram(img.flatten(), bins=256, range=(0,255))[0]
    probs = hist / np.sum(hist)
    probs = probs[probs > 0]
    return -np.sum(probs * np.log2(probs))

def correlation_coefficient(img):
    img = img.astype(np.float64)
    h = np.corrcoef(img[:, :-1].flatten(), img[:, 1:].flatten())[0,1]
    v = np.corrcoef(img[: -1, :].flatten(), img[1:, :].flatten())[0,1]
    d = np.corrcoef(img[: -1, :-1].flatten(), img[1:, 1:].flatten())[0,1]
    return h, v, d

# =====
# 4. LOAD DATA
# =====

data_dir = "C:/Users/hitha sunil/Desktop/DOTA/preprocessed/dota_resized"
image_files = [f for f in os.listdir(data_dir) if f.endswith(('.png', '.jpg', '.

ssim_scores = []
npcr_values = []
uaci_values = []
entropy_values = []
corr_original = []
corr_encrypted = []
enc_times = []
dec_times = []
euclidean_distances = []

features_original = []
features_decrypted = []
```

```
labels = []

# =====
# 5. PROCESS ALL IMAGES
# =====

for img_file in image_files:

    img_path = os.path.join(data_dir, img_file)
    image = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    image = cv2.resize(image, (128,128))

    # Encryption timing
    start_enc = time.time()
    encrypted_img, perm_indices, logistic_seq = encrypt_image(image)
    end_enc = time.time()

    # Decryption timing
    start_dec = time.time()
    decrypted_img = decrypt_image(encrypted_img, perm_indices, logistic_seq)
    end_dec = time.time()

    enc_times.append(end_enc - start_enc)
    dec_times.append(end_dec - start_dec)

    ssim_scores.append(compute_ssim(image, decrypted_img))
    npcr, uaci = compute_npcr_uaci(image, encrypted_img)
    npcr_values.append(npcr)
    uaci_values.append(uaci)
    entropy_values.append(image_entropy(encrypted_img))

    corr_original.append(correlation_coefficient(image))
    corr_encrypted.append(correlation_coefficient(encrypted_img))

    features_original.append(image.flatten())
    features_decrypted.append(decrypted_img.flatten())
    labels.append(1)
    labels.append(0)

    euclidean_distances.append(euclidean(image.flatten(), decrypted_img.flatte

# =====
# 6. PRINT STATISTICS
# =====

print("Processed:", len(image_files))
print("SSIM:", np.mean(ssim_scores), "±", np.std(ssim_scores))
print("NPCR:", np.mean(npcr_values), "±", np.std(npcr_values))
print("UACI:", np.mean(uaci_values), "±", np.std(uaci_values))
print("Entropy:", np.mean(entropy_values), "±", np.std(entropy_values))
print("Avg Encryption Time:", np.mean(enc_times))
print("Avg Decryption Time:", np.mean(dec_times))

# =====
# 7. CLASSIFICATION
# =====
```

```
X = np.array(features_original + features_decrypted)
y = np.array(labels)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

clf = SVC(kernel='linear')
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

print("Classification Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

# =====
# 8. VISUALIZATION PANELS
# =====

# SSIM Distribution
plt.figure()
sns.histplot(ssim_scores, kde=True)
plt.title("SSIM Distribution")
plt.xlabel("SSIM")
plt.show()

# Entropy Distribution
plt.figure()
sns.histplot(entropy_values, kde=True)
plt.title("Encrypted Image Entropy Distribution")
plt.xlabel("Entropy")
plt.show()

# Correlation Comparison
orig_corr_mean = np.mean(corr_original, axis=0)
enc_corr_mean = np.mean(corr_encrypted, axis=0)

labels_corr = ["Horizontal", "Vertical", "Diagonal"]
x = np.arange(3)

plt.figure()
plt.bar(x-0.2, orig_corr_mean, width=0.4, label="Original")
plt.bar(x+0.2, enc_corr_mean, width=0.4, label="Encrypted")
plt.xticks(x, labels_corr)
plt.title("Adjacent Pixel Correlation Comparison")
plt.legend()
plt.show()

# PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

plt.figure()
sns.scatterplot(x=X_pca[:,0], y=X_pca[:,1], hue=y)
plt.title("PCA Projection")
plt.show()

# t-SNE
```



```
tsne = TSNE(n_components=2, random_state=42)
X_tsne = tsne.fit_transform(X)

plt.figure()
sns.scatterplot(x=X_tsne[:,0], y=X_tsne[:,1], hue=y)
plt.title("t-SNE Projection")
plt.show()
```

Processed: 2806

SSIM: 0.9877591956503685 ± 0.025784631410013793

NPCR: 99.40855062951043 ± 0.13430589637392762

UACI: 26.267605605449372 ± 2.306073352646349

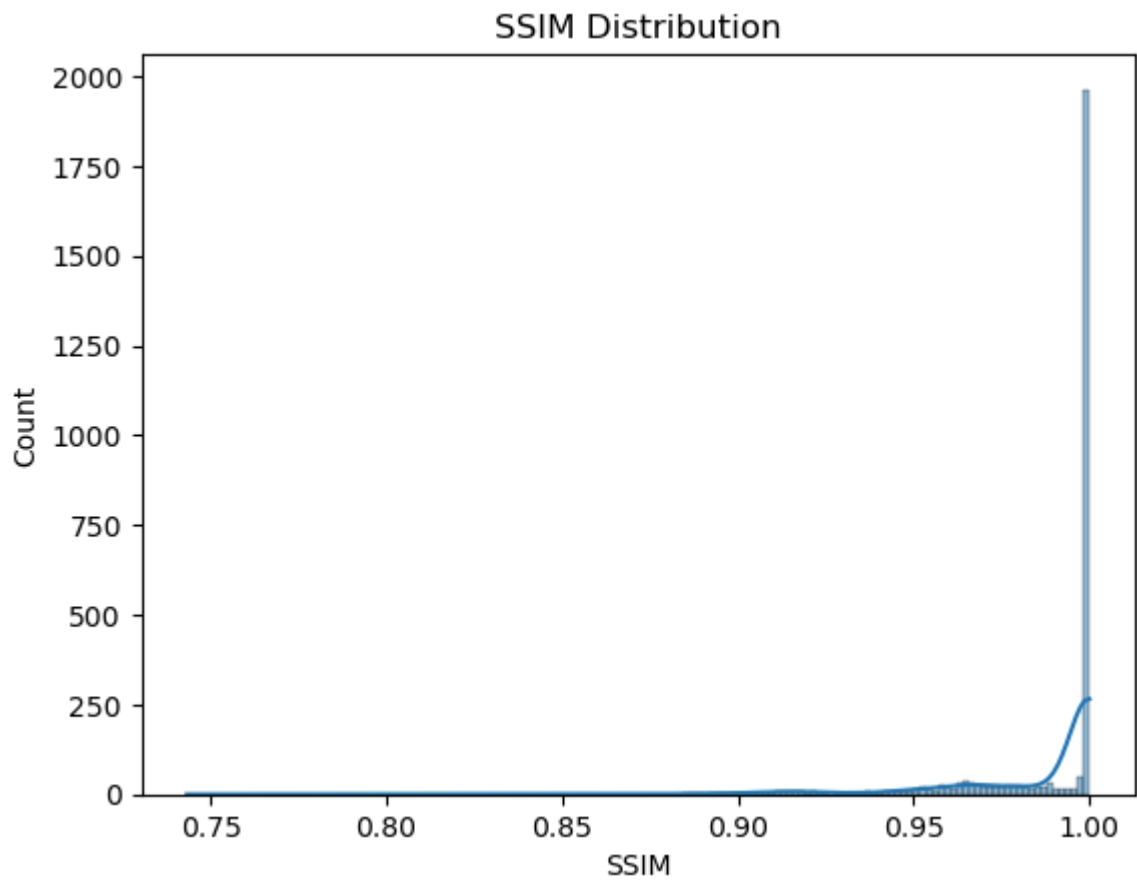
Entropy: 7.919096390712507 ± 0.03005281774013032

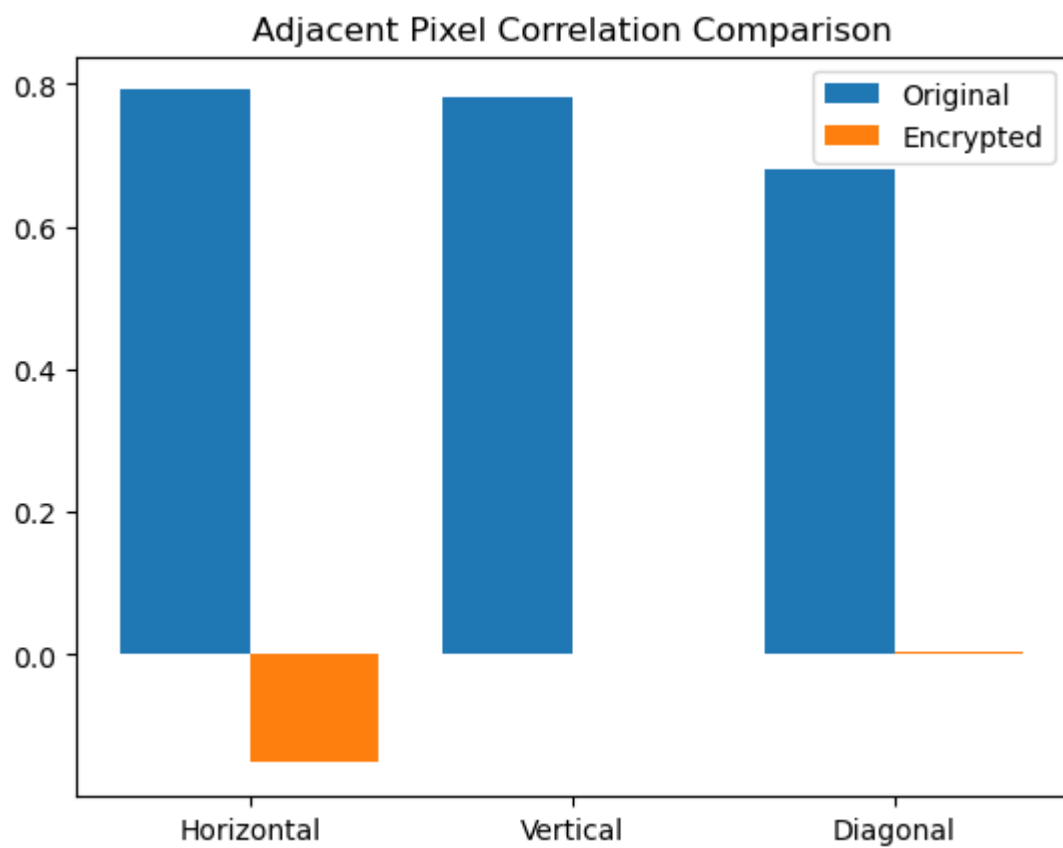
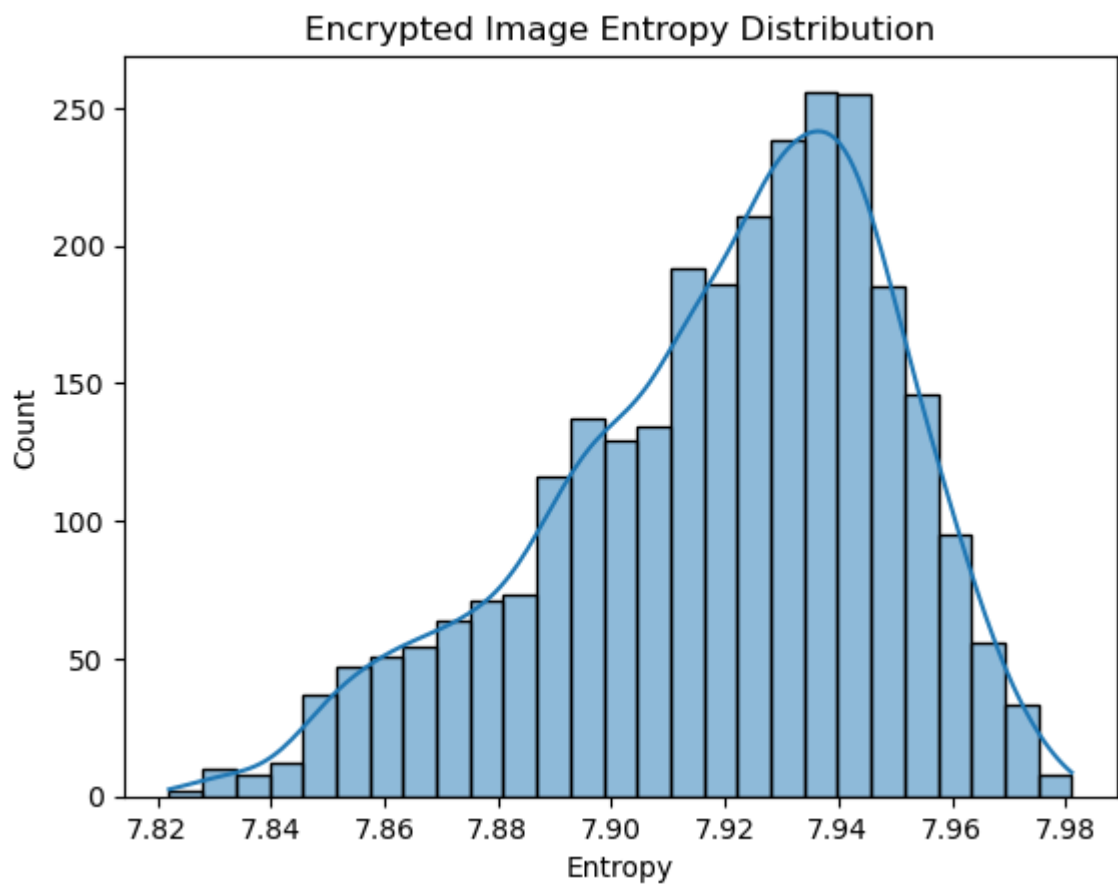
Avg Encryption Time: 0.03277816325532311

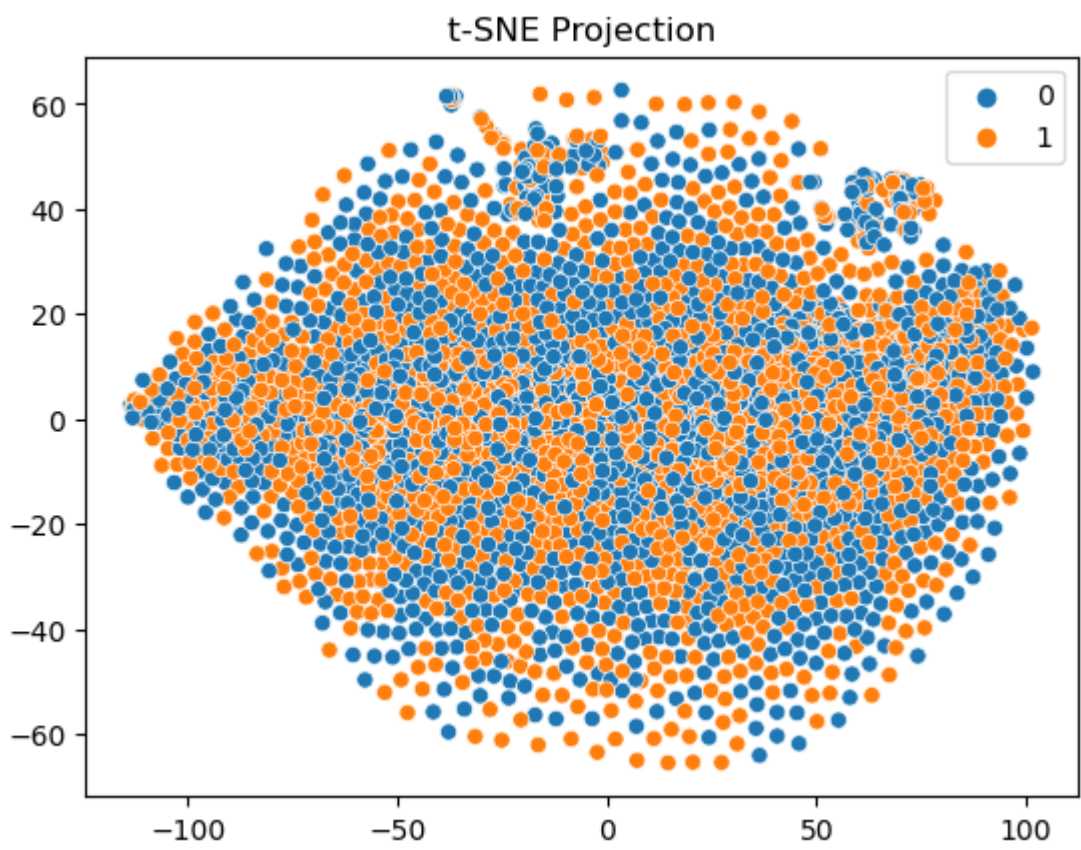
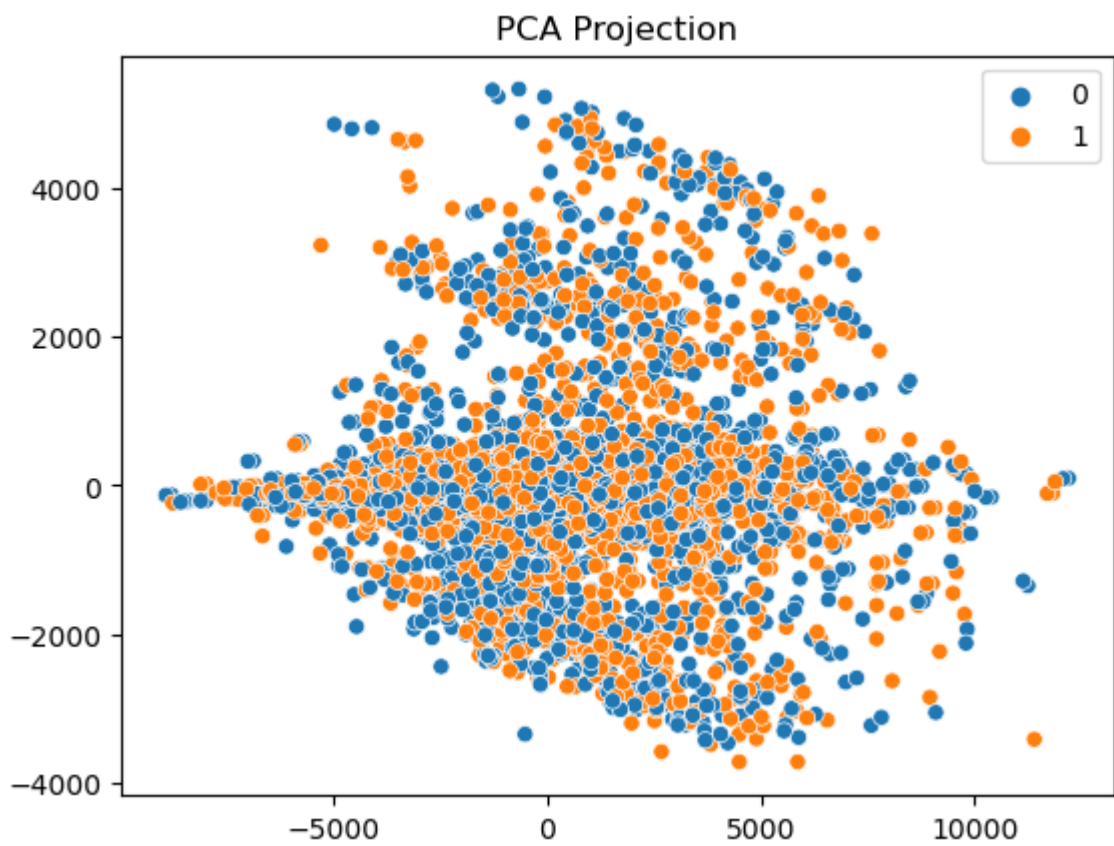
Avg Decryption Time: 0.0009983054927138032

Classification Accuracy: 0.9118432769367765

	precision	recall	f1-score	support
0	0.91	0.92	0.92	594
1	0.91	0.90	0.91	529
accuracy			0.91	1123
macro avg	0.91	0.91	0.91	1123
weighted avg	0.91	0.91	0.91	1123







In []: